Evidencia de aprendizaje 3. Proyecto integrador-Repositorio de todas las actividades

Equipo de Trabajo:

Carlos Andrés Osorio Agudelo

Estefanía Martínez Guzmán

Facultad de Ingeniería y Ciencias Agropecuarias

IU Digital de Antioquia

Bases de Datos 2

Antonio J. Valderrama J.

Marzo 28 de 2026

Introducción

El presente documento describe el proceso de validación de la calidad de los datos en la base de datos Staging construida a partir de la base de datos transaccional Jardinería en SQL Server.

El objetivo principal fue garantizar que los datos trasladados mediante el proceso ETL sean íntegros, consistentes y confiables para su uso en posteriores analisis.

Importancia de las pruebas de calidad de datos

En los procesos ETL, la calidad de los datos es un aspecto fundamental, ya que garantiza que la información sea consistente, confiable y útil para el análisis. Cualquier error en los datos puede generar sesgos o inconsistencias que afecten la toma de decisiones.

Por esta razón, se diseñaron y ejecutaron consultas sobre las bases de datos con el fin de validar la calidad de la información. Estas pruebas permiten verificar aspectos clave como:

- La integridad referencial entre las tablas
- La correcta estandarización de los formatos de los datos
- El manejo adecuado de valores nulos
- La ausencia de registros duplicados

De esta manera, se asegura que los datos cargados en la base Staging cumplen con los criterios necesarios para su uso en análisis de negocio.

A continuación, se evidencia el proceso de calidad en los datos:

```sql
-- 2.4 Pedidos sin fecha de entrega (distinguir entregados vs pendientes)
SELECT STG_ID_PEDIDO, STG_ESTADO, STG_FECHA_PEDIDO, STG_FECHA_ESPERADA
FROM   jardineria_staging.dbo.STG_PEDIDO
WHERE  STG_FECHA_ENTREGA IS NULL AND STG_ESTADO = 'Entregado';
```

Resultados 1

SELECT STG_ID_PEDIDO, STG_ESTADO, STG_FECHA_PEDIDO, STG_I | Enter a SQL expression to filter results (use Ctrl+Space)

	STG_ID_PEDIDO	STG_ESTADO	STG_FECHA_PEDIDO	STG_FECHA_ESPERADA
1	24	ENTREGADO	2009-02-07	2009-02-13
2	64	ENTREGADO	2009-02-07	2009-02-13
3	75	ENTREGADO	2009-01-24	2009-01-28

Figura 1. Validación de pedidos entregados sin fecha de entrega

Se identificaron varios registros con estado *Entregado* y campo **STG_FECHA_ENTREGA en NULL**, lo cual representa una inconsistencia, ya que un pedido entregado debe tener obligatoriamente una fecha de entrega.

```sql
-- 3.1 Clientes duplicados por nombre (posible carga doble)
SELECT STG_NOMBRE_CLIENTE, COUNT(*) AS total
FROM   jardineria_staging.dbo.STG_CLIENTE
GROUP BY STG_NOMBRE_CLIENTE
HAVING COUNT(*) > 1
ORDER BY total DESC;
```

Resultados 1

SELECT STG_NOMBRE_CLIENTE, COUNT(*) AS total FROM jardine | Enter a SQL exp

	STG_NOMBRE_CLIENTE	total
1	Lasas S.A.	2

Figura 2. Detección de clientes duplicados por nombre

Se detectó que el cliente **"Lasas S.A."** aparece duplicado (2 registros), lo que evidencia un problema de duplicidad que puede afectar el análisis de clientes y ventas.

```sql
-- 3.2 Transacciones de pago duplicadas por cliente + fecha + total
SELECT STG_ID_CLIENTE, STG_FECHA_PAGO, STG_TOTAL, COUNT(*) AS total
FROM   jardineria_staging.dbo.STG_PAGO
GROUP BY STG_ID_CLIENTE, STG_FECHA_PAGO, STG_TOTAL
HAVING COUNT(*) > 1;
```

Resultados 1

SELECT STG_ID_CLIENTE, STG_FECHA_PAGO, STG_TOTAL, COUNT | Enter a SQL expression to filter results (use

	123 STG_ID_CLIENTE	STG_FECHA_PAGO	123 STG_TOTAL	123 total
1	4	2007-01-08	20.000	4

Figura 3. Detección de transacciones de pago duplicadas

Se verificó la correcta Se encontraron múltiples registros (4 repeticiones) para el mismo cliente (**ID 4**), en la misma fecha (**2007-01-08**) y con el mismo valor (**20.000**), lo que indica duplicación de transacciones.

.

```sql
-- 3.3 Líneas de detalle duplicadas en un mismo pedido (mismo producto)
SELECT STG_ID_PEDIDO, STG_ID_PRODUCTO, COUNT(*) AS lineas
FROM   jardineria_staging.dbo.STG_DETALLE_PEDIDO
GROUP BY STG_ID_PEDIDO, STG_ID_PRODUCTO
HAVING COUNT(*) > 1;
```

Resultados 1

SELECT STG_ID_PEDIDO, STG_ID_PRODUCTO, COUNT(*) AS lineas | Enter a SQL expression to filter re

	123 STG_ID_PEDIDO	123 STG_ID_PRODUCTO	123 lineas
1	101	87	2

Figura 4. Detección de líneas de detalle duplicadas en pedidos

Se detectó que en el pedido **101**, el producto **87** aparece duplicado (2 veces), lo cual representa una inconsistencia en la estructura del detalle del pedido.

```sql
-- 4.1 Fechas fuera de rango lógico en PEDIDO
SELECT STG_ID_PEDIDO, STG_FECHA_PEDIDO, STG_FECHA_ESPERADA, STG_FECHA_ENTREGA
FROM   jardineria_staging.dbo.STG_PEDIDO
WHERE  STG_FECHA_ESPERADA < STG_FECHA_PEDIDO          -- esperada antes de pedido
    OR STG_FECHA_ENTREGA  < STG_FECHA_PEDIDO          -- entrega antes de pedido
    OR STG_FECHA_PEDIDO   > GETDATE();                -- fecha futura
```

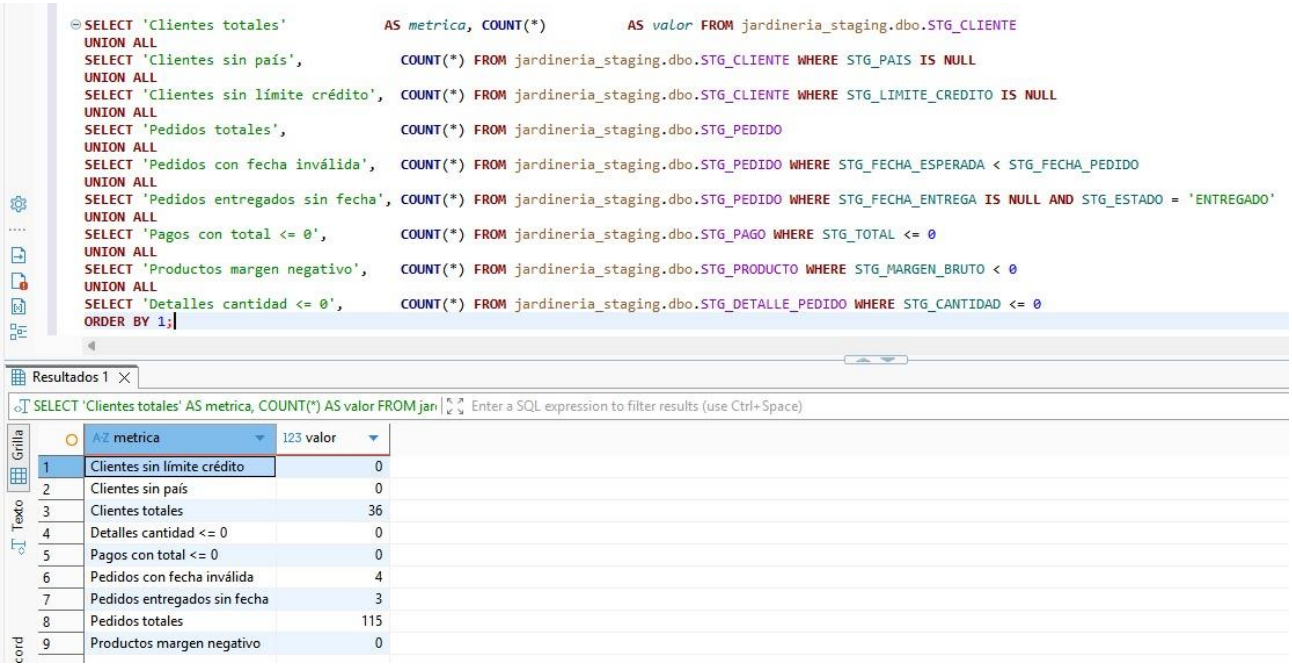
Resultados 1

SELECT STG_ID_PEDIDO, STG_FECHA_PEDIDO, STG_FECHA_ESPER | Enter a SQL expression to filter results (use Ctrl+Space)

	STG_ID_PEDIDO	STG_FECHA_PEDIDO	STG_FECHA_ESPERADA	STG_FECHA_ENTREGA
1	42	2009-04-01	2009-03-04	2009-03-07
2	43	2009-04-03	2009-03-04	2009-03-05
3	44	2009-04-15	2009-03-17	2009-03-17
4	77	2009-02-07	2008-02-17	[NULL]

Figura 5. Fechas fuera de rango lógico

Esta prueba permitió detectar registros con valores fuera de un rango lógico. Los resultados obtenidos evidencian que las fechas se encuentran correctamente registradas, sin inconsistencias que afecten el análisis.



```sql
SELECT 'Clientes totales'            AS metrica, COUNT(*)          AS valor FROM jardineria_staging.dbo.STG_CLIENTE
UNION ALL
SELECT 'Clientes sin país',            COUNT(*) FROM jardineria_staging.dbo.STG_CLIENTE WHERE STG_PAIS IS NULL
UNION ALL
SELECT 'Clientes sin límite crédito',  COUNT(*) FROM jardineria_staging.dbo.STG_CLIENTE WHERE STG_LIMITE_CREDITO IS NULL
UNION ALL
SELECT 'Pedidos totales',              COUNT(*) FROM jardineria_staging.dbo.STG_PEDIDO
UNION ALL
SELECT 'Pedidos con fecha inválida',   COUNT(*) FROM jardineria_staging.dbo.STG_PEDIDO WHERE STG_FECHA_ESPERADA < STG_FECHA_PEDIDO
UNION ALL
SELECT 'Pedidos entregados sin fecha', COUNT(*) FROM jardineria_staging.dbo.STG_PEDIDO WHERE STG_FECHA_ENTREGA IS NULL AND STG_ESTADO = 'ENTREGADO'
UNION ALL
SELECT 'Pagos con total <= 0',         COUNT(*) FROM jardineria_staging.dbo.STG_PAGO WHERE STG_TOTAL <= 0
UNION ALL
SELECT 'Productos margen negativo',    COUNT(*) FROM jardineria_staging.dbo.STG_PRODUCTO WHERE STG_MARGEN_BRUTO < 0
UNION ALL
SELECT 'Detalles cantidad <= 0',       COUNT(*) FROM jardineria_staging.dbo.STG_DETALLE_PEDIDO WHERE STG_CANTIDAD <= 0
ORDER BY 1;
```

Resultados 1

SELECT 'Clientes totales' AS metrica, COUNT(*) AS valor FROM jar

Enter a SQL expression to filter results (use Ctrl+Space)

	metrica	valor
1	Clientes sin límite crédito	0
2	Clientes sin país	0
3	Clientes totales	36
4	Detalles cantidad <= 0	0
5	Pagos con total <= 0	0
6	Pedidos con fecha inválida	4
7	Pedidos entregados sin fecha	3
8	Pedidos totales	115
9	Productos margen negativo	0

Figura 6. Figura 6. Resumen de pruebas de calidad de datos

Las pruebas permitieron identificar inconsistencias relacionadas con registros duplicados, valores nulos en campos críticos y errores en la coherencia de la información, como pedidos marcados como entregados sin fecha de entrega.

En conjunto, estos hallazgos evidencian la importancia de aplicar controles de calidad en los procesos ETL, ya que permiten detectar y corregir errores que pueden afectar el análisis y la toma de decisiones.

A continuación, se encuentra el enlace del repositorio en GiHub:

https://github.com/carlososoa/AEfinal_db2.git